bucles python.pdf

ifesther

Fundamentos de Programación

1° Grado en Sistemas de Información

Escuela Politécnica Superior
Universidad de Alcalá



Reservados todos los derechos.
No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.


Elijas lo que elijas es un 10 asegurado
coches.net
Un coche de renting te permite ser indeciso sin tener que hacer una inversión a largo plazo.
NO LO OLVIDES
DILO JUAN, DILO: REN-TING
coches.net

CONDICIONALES

En Python, las estructuras condicionales if, else, y elif se utilizan para tomar decisiones basadas en condiciones específicas. Aquí te explico cómo funcionan:

- **if** (si): La declaración if se utiliza para evaluar una condición. Si la condición es verdadera (True), el bloque de código dentro del if se ejecuta. Si la condición es falsa (False), el bloque de código se omite.

```
edad = 18
if edad >= 18:
    print("Eres mayor de edad.")
```

- **else** (si no): La declaración else se utiliza para proporcionar una alternativa al bloque if. Si la condición del if es falsa, el bloque de código dentro del else se ejecuta.

```
edad = 15
if edad >= 18:
    print("Eres mayor de edad.")
else:
    print("Eres menor de edad.")
```

- **elif** (sino, si): La declaración elif (abreviatura de "else if") se utiliza para evaluar múltiples condiciones secuenciales. Puedes tener múltiples bloques elif después de un if y antes de un else. El primer bloque elif cuya condición sea verdadera se ejecutará.

```
nota = 75
if nota >= 90:
    print("Tienes una A.")
elif nota >= 80:
    print("Tienes una B.")
elif nota >= 70:
    print("Tienes una C.")
else:
    print("Tienes una F.")
```

En el ejemplo anterior, Python evaluará las condiciones de arriba hacia abajo, y tan pronto como una condición sea verdadera, ejecutará el bloque de código correspondiente y saldrá del condicional. Si ninguna de las condiciones es verdadera, se ejecutará el bloque else (si está presente).

Las estructuras condicionales if, else, y elif son fundamentales para controlar el flujo de un programa en Python y tomar decisiones basadas en condiciones específicas.

WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

BUCLES FOR

Los bucles for en Python se utilizan para iterar sobre una secuencia de elementos, como una lista, una tupla, una cadena de texto o un rango de números. La estructura básica de un bucle ***for*** es la siguiente:

```
for elemento in secuencia:
    # Código que se ejecuta en cada iteración
```

1. Iterar sobre una **lista** de números:

```
numeros = [1, 2, 3, 4, 5]
for numero in numeros:
    print(numero)
```

Este bucle imprimirá los números del 1 al 5 en la consola.

2. Iterar sobre una **cadena** de texto:

```
mensaje = "Hola, mundo!"
for letra in mensaje:
    print(letra)
```

Este bucle imprimirá cada letra del mensaje en la consola.

3. Iterar sobre un **rango** de números:

```
for i in range(1, 6):
    print(i)
```

Este bucle imprimirá los números del 1 al 5 utilizando la función range().

4. Iterar sobre una lista de elementos y realizar una tarea **condicional**:

```
frutas = ["manzana", "plátano", "uva", "pera", "fresa"]
fruta_a_buscar = "uva"
encontrada = False

for fruta in frutas:
    if fruta == fruta_a_buscar:
        encontrada = True
        break

if encontrada:
    print(f"{fruta_a_buscar} encontrada en la lista.")
else:
    print(f"{fruta_a_buscar} no encontrada en la lista.")
```

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH

Dilo Juan, dilo: Ren-ting. No lo olvides. Elijas lo que elijas es un 10 asegurado - coches.net

coches.net

DILO JUAN, DILO: REN-TING

NO LO OLVIDES

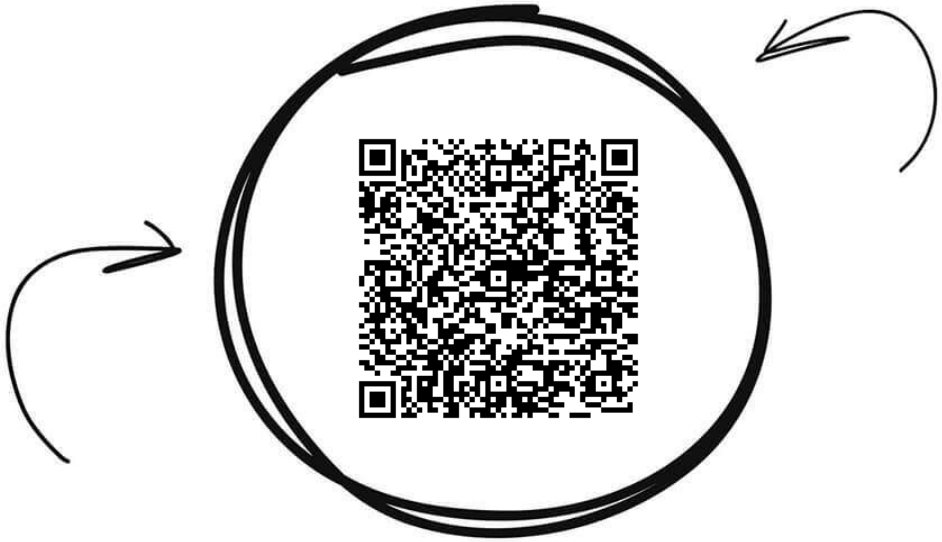
Elijas lo que elijas es un 10 asegurado



coches.net

Un coche de renting te permite ser indeciso sin tener que hacer una inversión a largo plazo.

Fundamentos de Programación



Banco de apuntes de la

WUOLAH

Comparte estos flyers en tu clase y consigue más dinero y recompensas

1. Imprime esta hoja
2. Recorta por la mitad
3. Coloca en un lugar visible para que tus compis puedan escanar y acceder a apuntes
4. Llévate dinero por cada descarga de los documentos descargados a través de tu QR

En este ejemplo, el bucle for busca una fruta específica en la lista y establece la variable encontrada en True si la encuentra. Después, se imprime un mensaje según si la fruta se encontró o no.

Los bucles for son una herramienta poderosa para recorrer secuencias y realizar tareas repetitivas en Python. Puedes personalizarlos según tus necesidades específicas, lo que los hace muy versátiles.

BUCLES WHILE

La función while en Python se utiliza para crear bucles o ciclos que ejecutan un bloque de código repetidamente mientras una condición especificada sea verdadera. La estructura básica de un bucle while es la siguiente:

while condición:
    # Código que se ejecuta mientras la condición sea verdadera

Cómo funciona: El bucle while comienza evaluando la condición. Si la condición es verdadera, se ejecuta el bloque de código que sigue al while. Luego, la condición se vuelve a evaluar. Este proceso continúa hasta que la condición se vuelva falsa. Cuando la condición es falsa, el bucle while se detiene, y la ejecución del programa continúa con las instrucciones después del bucle.

Para qué se usa: Los bucles while son útiles cuando no sabes de antemano cuántas veces debes repetir un bloque de código, ya que depende de una condición específica. Son ideales para tareas como recorrer una lista o realizar cálculos hasta que se cumpla una cierta condición. Sin embargo, es importante tener cuidado para evitar bucles infinitos, donde la condición nunca se vuelve falsa, lo que podría bloquear el programa.

```
contador = 0
while contador < 5:
    print(f"Contador: {contador}")
    contador += 1
```

Este bucle while imprimirá "Contador: 0", "Contador: 1", "Contador: 2", "Contador: 3", "Contador: 4" en la consola. El contador se incrementa en cada iteración, y el bucle se detiene cuando el contador alcanza 5 y la condición contador < 5 ya no es verdadera.

1. Búsqueda de Raíces:

Supongamos que queremos encontrar la raíz cuadrada de un número dado utilizando el método de Newton. Un bucle while puede refinar iterativamente tu estimación de la raíz hasta que alcances una precisión deseada. El bucle se ejecuta hasta que la diferencia entre la estimación actual y la estimación anterior sea menor que un valor específico.

```
objetivo = 16
estimacion = 4.0
precision = 1e-6
```

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH

Dilo Juan, dilo: Ren-ting. No lo olvides. Elijas lo que elijas es un 10 asegurado - coches.net


Elijas lo que elijas es un 10 asegurado
coches.net
Un coche de renting te permite ser indeciso sin tener que hacer una inversión a largo plazo.
NO LO OLVIDES
DILO JUAN, DILO: REN-TING
coches.net

```
while abs(estimacion * estimacion - objetivo) > precision:
    estimacion = 0.5 * (estimacion + objetivo / estimacion)

print(f"La raíz cuadrada de {objetivo} es aproximadamente {estimacion}")
```

2. : Simulación de Juego:

Imagina que estás creando un juego de rol y quieres simular una batalla en la que los personajes se atacan mutuamente hasta que uno de ellos queda sin puntos de vida (HP). Puedes usar un bucle while para controlar la simulación de la batalla hasta que uno de los personajes pierda todos sus HP.

```
jugador1_hp = 100
jugador2_hp = 120

while jugador1_hp > 0 and jugador2_hp > 0:
    # Simular un turno de la batalla
    # Restar HP, calcular daño, etc.

if jugador1_hp <= 0:
    print("Jugador 2 gana la batalla.")
else:
    print("Jugador 1 gana la batalla.")
```

3. Procesamiento de Datos en Lote:

Imagina que estás procesando una gran cantidad de datos almacenados en un archivo y quieres procesarlos en lotes de, por ejemplo, 100 registros a la vez. Puedes usar un bucle while para procesar los datos en lotes hasta que hayas procesado todo el archivo.

```
datos = cargar_datos_desde_archivo()
lote_actual = []
lote_tamaño = 100

while datos:
    lote_actual = datos[:lote_tamaño]
    datos = datos[lote_tamaño:]
    procesar_lote(lote_actual)
```

SUBPROGRAMAS

Los subprogramas, también conocidos como funciones o procedimientos, son bloques de código independientes que realizan tareas específicas en un programa más grande. Se utilizan para modularizar y organizar el código, mejorar la legibilidad y reutilizar funcionalidad en múltiples partes del programa. Aquí tienes una explicación más detallada sobre cómo funcionan, para qué se usan, cómo se usan, sus ventajas y las funciones que cumplen:

WUOLAH
Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

Cómo funcionan:
Un subprograma es una entidad separada que toma uno o más valores de entrada (parámetros), realiza un conjunto de instrucciones y devuelve un resultado o efectúa una acción. El programa principal llama al subprograma cuando necesita realizar una tarea específica.
Los subprogramas pueden ser funciones (que devuelven un valor) o procedimientos (que realizan acciones sin devolver un valor).

Para qué se usan:
- **Modularidad**: Los subprogramas dividen el programa en partes más pequeñas y manejables, lo que facilita la resolución de problemas y el mantenimiento del código.
- **Reutilización**: Puedes usar el mismo subprograma en diferentes partes del programa, evitando duplicar código.
- **Abstracción**: Los subprogramas ocultan los detalles internos de su implementación, permitiendo a otros desarrolladores utilizarlos sin necesidad de entender cómo funcionan internamente.

Cómo se usan:
- Declaración: Los subprogramas se declaran antes de su uso, especificando su nombre, parámetros, tipo de retorno (en el caso de funciones) y cuerpo.
- Invocación: Se llaman al utilizar su nombre y proporcionando los valores de entrada (si los tienen). Si es una función, el resultado se puede usar en expresiones; si es un procedimiento, realiza acciones sin devolver un valor.

Ventajas:
- Modularidad: Facilita la división del código en partes manejables y fáciles de entender.
- Reutilización: Evita la duplicación de código y mejora la eficiencia del desarrollo.
- Mantenibilidad: Permite hacer cambios en una parte del programa sin afectar otras áreas.
- Abstracción: Oculta detalles de implementación y mejora la claridad del código.
- Depuración: Facilita la identificación y corrección de errores.

Funciones:
Funciones: Devuelven un valor calculado y son útiles para realizar cálculos y procesamiento de datos.
Procedimientos: Realizan acciones sin devolver un valor y son útiles para modificar el estado del programa o realizar tareas sin salida específica.

Ejemplo:
1. Cálculo de Impuestos:

Subprograma: calcular_impuestos(total_ingresos)
Función que toma el total de ingresos como parámetro y calcula el monto de impuestos a pagar en función de las tasas impositivas vigentes.
Subprograma: mostrar_resultado(total_ingresos, impuestos)
Procedimiento que muestra el resultado en la pantalla.

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH

Dilo Juan, dilo: Ren-ting. No lo olvides. Elijas lo que elijas es un 10 asegurado - coches.net

Programa Principal:

ingresos = 50000
impuestos = calcular_impuestos(ingresos)
mostrar_resultado(ingresos, impuestos)
Ejemplo 2: Procesamiento de Datos de Empleados:

Subprograma: calcular_salario_anual(salario_mensual)
Función que toma el salario mensual de un empleado y calcula su salario anual.
Subprograma: generar_reporte_empleados(lista_empleados)
Procedimiento que toma una lista de empleados y genera un informe detallado con sus salarios anuales.
Programa Principal:
python
Copy code

```
empleados = [ ... ]  # Lista de empleados
for empleado in empleados:
    salario_anual = calcular_salario_anual(empleado.salario_mensual)
    empleado.salario_anual = salario_anual
generar_reporte_empleados(empleados)
```

Ejemplo 3: Simulación de Juego de Rol:

Subprograma: iniciar_batalla(jugador1, jugador2)
Procedimiento que simula una batalla entre dos jugadores y actualiza sus puntos de vida.
Subprograma: verificar_ganador(jugador1, jugador2)
Función que determina si hay un ganador en la batalla y devuelve el resultado.
Programa Principal:
python
Copy code

```
jugador1 = crear_personaje("Alicia", 100)
jugador2 = crear_personaje("Bob", 120)
while not verificar_ganador(jugador1, jugador2):
    iniciar_batalla(jugador1, jugador2)
ganador = verificar_ganador(jugador1, jugador2)
print(f"{ganador.nombre} gana la batalla.")
```

En estos ejemplos, se utilizan subprogramas para realizar tareas específicas, como cálculos de impuestos, procesamiento de datos de empleados y simulación de una batalla en un juego de rol. Los subprogramas ayudan a organizar y modularizar el código principal, facilitando su comprensión y mantenimiento.

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH

Dilo Juan, dilo: Ren-ting. No lo olvides. Elijas lo que elijas es un 10 asegurado - coches.net